SinghArpit2492 Chess Project

C# Chess Engine Walkthrough

Chess Engine Walkthrough

I have completed the entire C# chess engine from scratch. The code is fully original (not copied from Sebastian Lague) and features modern C# formatting, compact ushort move encoding, iterative deepening, alpha-beta pruning minimax search, quiescence search, and a beautiful console-based interactive UI with legal move highlighting!

File Structure & Components Written

All files have been successfully created in the workspace under the `d:\Chess\` directory:

1. Setup & Project Configuration

- Chess.sln — Visual Studio / VS Code solution file.
- ChessEngine.csproj — Minimal .NET 8 console application project configuration.

2. Core Chess Representation (`ChessEngine.Core`)

- Piece.cs — Defines piece types, color flags, and helper functions (e.g., bitwise encoding).
- Coord.cs — Handles conversion between a1-h8 algebraic notation and 0-63 square indexing.
- Move.cs — Encodes moves in a compact 16-bit `ushort` structure (6 bits for origin, 6 bits for target, 4 bits for flags).
- GameState.cs — C# record saving captured pieces, castling flags, en passant, halfmove clock, and Zobrist hash, pushed onto the undo history stack.
- Board.cs — The main game state board. Implements `ExecuteMove` and `UndoMove` using Zobrist hash updates.
- FenParser.cs — Parses and exports FEN strings to represent board states.
- Zobrist.cs — Manages Zobrist hashing keys for hashing board states for transposition table cache.

3. Move Generation (`ChessEngine.MoveGeneration`)

- BoardHelper.cs — Caches static board topology (edge distances, pawn attacks, knight jumps, king target offsets).
- MoveGenerator.cs — Generates all legal moves, filtering pseudo-legal moves by checking if the resulting board leaves the friendly king in check.

4. Search and AI Logic (`ChessEngine.Search`)

- MoveRanker.cs — Orders moves descendingly using MVV-LVA (Most Valuable Victim - Least Valuable Attacker) and promotion weights to maximize Alpha-Beta pruning cutoffs.

- TranspositionTable.cs — Thread-safe Zobrist hash table cache avoiding duplicate evaluations.
- SearchEngine.cs — Implements Iterative Deepening, Minimax with Alpha-Beta pruning, and Quiescence Search (captures-only search to avoid the horizon effect).

5. Console UI & Entry Point

- Display.cs — Beautiful terminal renderer utilizing ANSI background colors and Unicode chess symbols. Highlighting is applied to selected pieces (DarkYellow) and their legal target moves (DarkGreen).
- Program.cs — The interactive game loop supporting coordinate selection, algebraic moves, game restart, check/checkmate detection, stalemate, and move undo.

Interactive Gameplay Features

• **Move Highlighting**: Type just the origin coordinate (e.g. `e2`) and press Enter to highlight all legal destination squares on the board in Green!

• **Move Entry**: Type the destination coordinate (e.g. `e4`) or enter the full move (e.g. `e2e4`) to execute the move.

• **Undo**: Type `undo` to immediately roll back both your move and the AI's move!

• **Restart**: Type `restart` to reset the board back to the initial starting position.

How to Compile and Play

■ *Important: The .NET 8 SDK is not yet installed on your system. Follow these quick steps to set it up:*

• **Download and Install .NET 8 SDK**:

Visit: https://dotnet.microsoft.com/en-us/download/dotnet/8.0

*(Make sure to download the **SDK** installer for Windows, not just the Runtime).*

• **Verify Installation**:

Open a terminal and run:

```
dotnet --version
```

• **Navigate to the Project and Run**:

Run the following commands to compile and launch the game:

```
cd d:\Chess
dotnet run --project ChessEngine
```